

# 机器视觉技术

孙明竹 刘晓芳

[sunmz@nankai.edu.cn](mailto:sunmz@nankai.edu.cn)

[liuxiaofang@nankai.edu.cn](mailto:liuxiaofang@nankai.edu.cn)



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南开大学

Nankai University

## 第二部分 图像处理基础

Image Processing

# 第3章 图像预处理

- 图像预处理在图像最低抽象层次上进行操作，处理的输入和输出都是亮度图像
- 图像预处理的目的是改善图像数据，抑制不需要的变形或者增强某些对于后续处理重要的图像特征



图像  
预处理

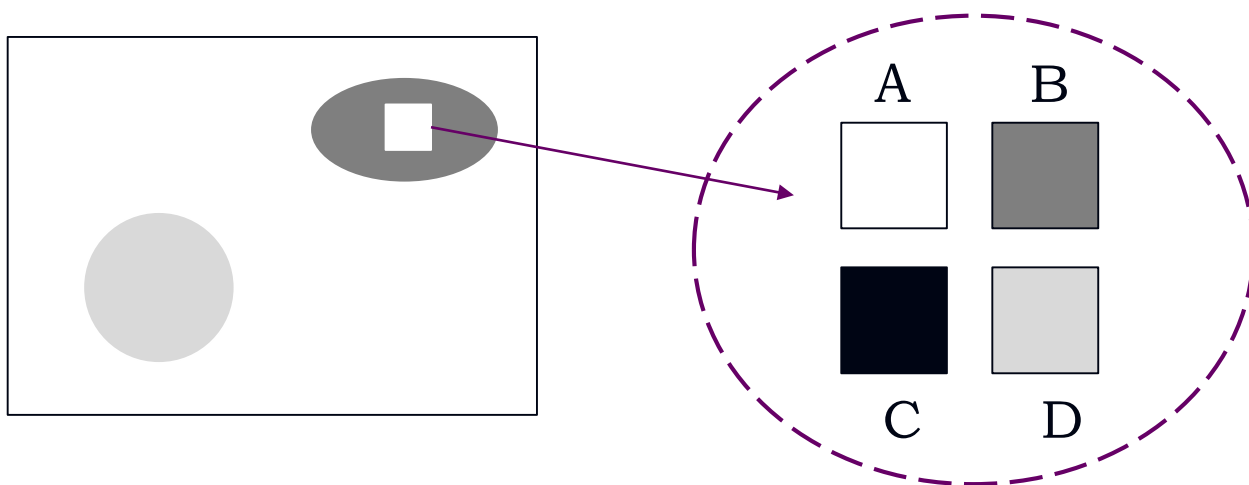


自动完成!

# 第3章 图像预处理

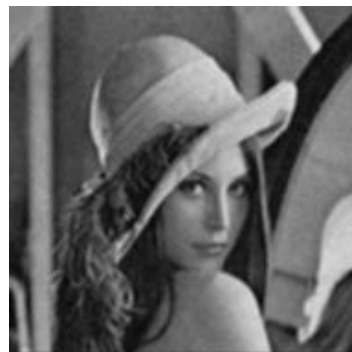


- 图像预处理在图像最低抽象层次上进行操作，处理的输入和输出都是亮度图像
- 图像预处理的目的是改善图像数据，抑制不需要的变形或者增强某些对于后续处理重要的图像特征
- 图像中存在相当多的冗余信息，实际图像中属于一个目标的相邻像素基本上具有相同的或类似的亮度值



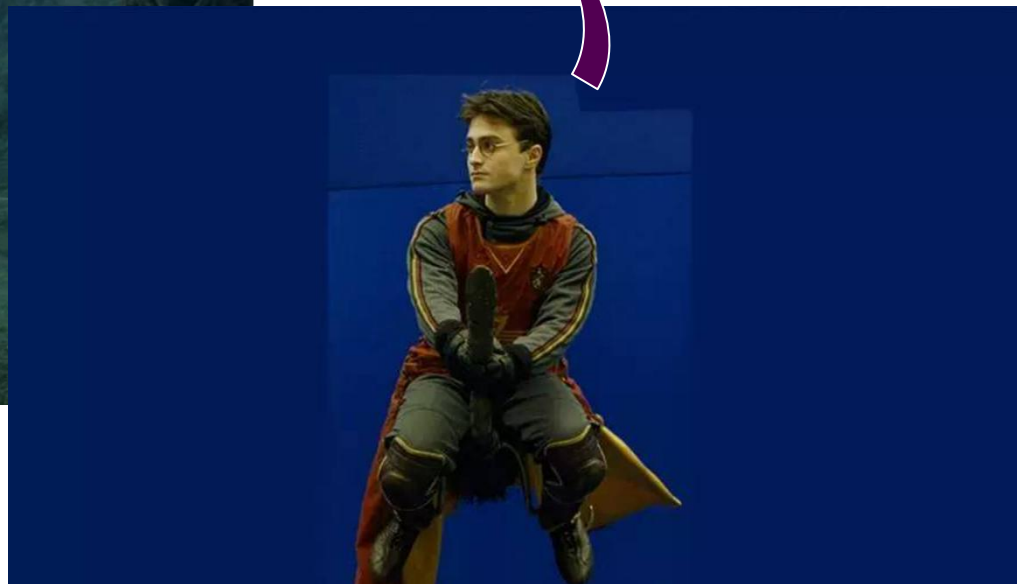
# 主要内容

- 第2章 数字图像的概念与性质
- 第3章 图像预处理
  - 3.1 像素亮度变换
  - 3.2 几何变换
  - 3.3 图像平滑
  - 3.4 边缘检测（编程内容，2课时）
- 第4章 二值图像处理
- 第5章 传统特征检测



## 3.1 像素亮度变换

- 像素亮度变换修改像素的亮度值，变换只取决于像素自身的性质



## 3.1 像素亮度变换

- 像素亮度变换修改像素的亮度值，变换只取决于像素自身的性质
- 亮度校正
  - 与像素位置相关，在改变像素的亮度时要考虑该像素原来的亮度及其在图像中的位置
- 灰度级变换
  - 与像素位置无关，在改变像素的亮度时无需考虑其在图像中的位置

## 3.1.1 亮度校正

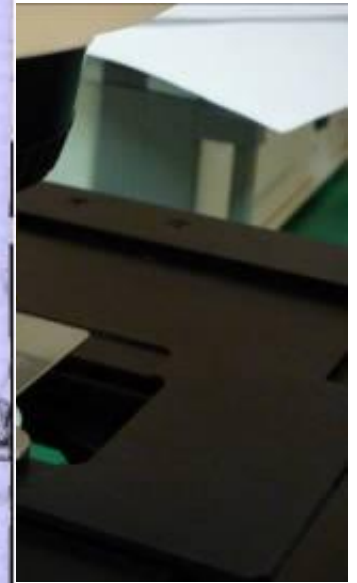
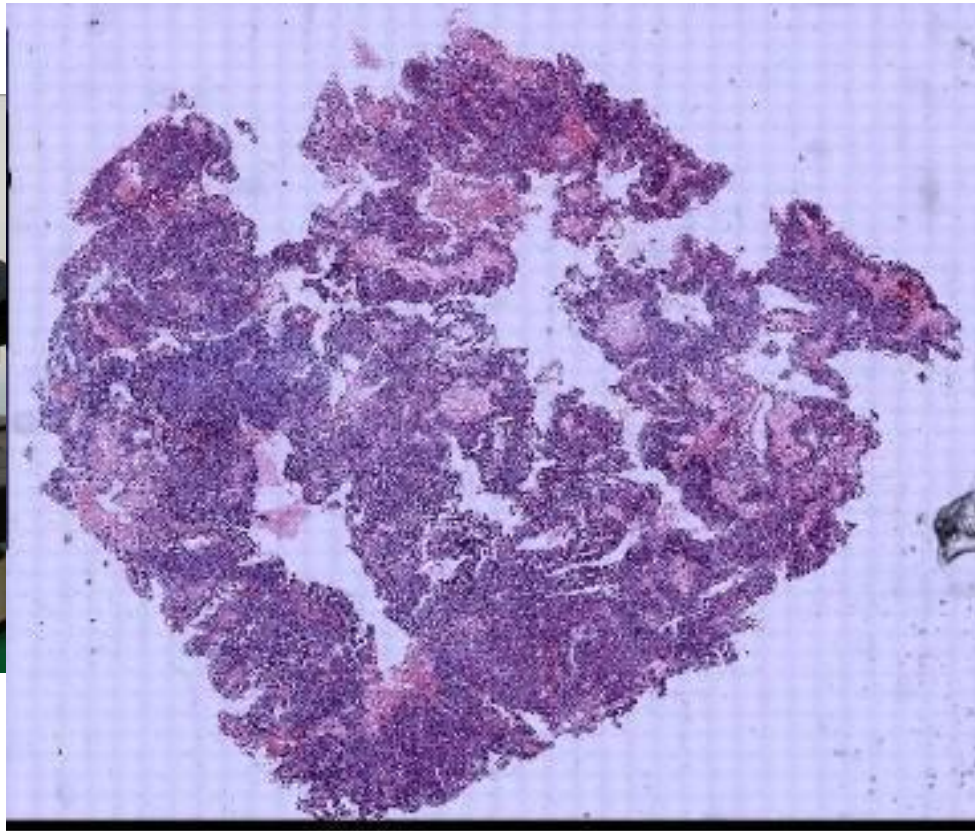
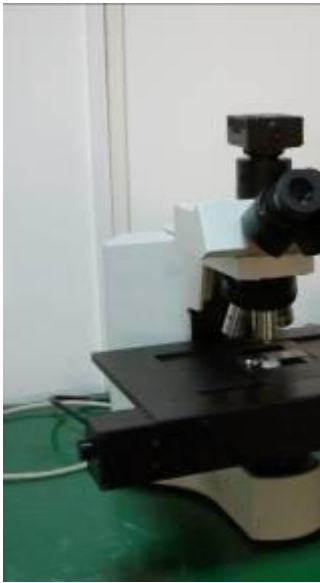
- 需要进行亮度校正的原因
  - 在远离光轴的位置，透镜对光线的削弱作用加强
  - 传感器的光敏元件在不同位置的灵敏度不同
  - 不均匀的物体照明
- 理想图像  $f(m,n)$ ，存在乘性错误系数  $e(m,n)$ ，退化后的图像： $g(m,n) = f(m,n)e(m,n)$

若得到已知亮度的参考图像  $g_c(m,n)$ ，最简单的情况，原理想参考图像具有不变的亮度  $c$ ，则可获得错误系数，进而完成亮度校正：

$$f(m,n) = \frac{g(m,n)}{e(m,n)} = \frac{cg(m,n)}{g_c(m,n)}$$

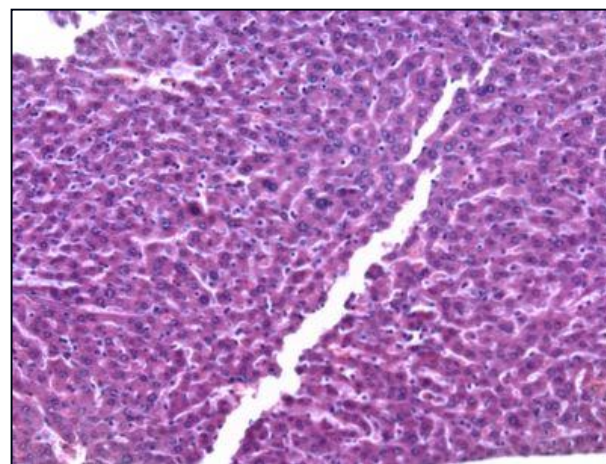
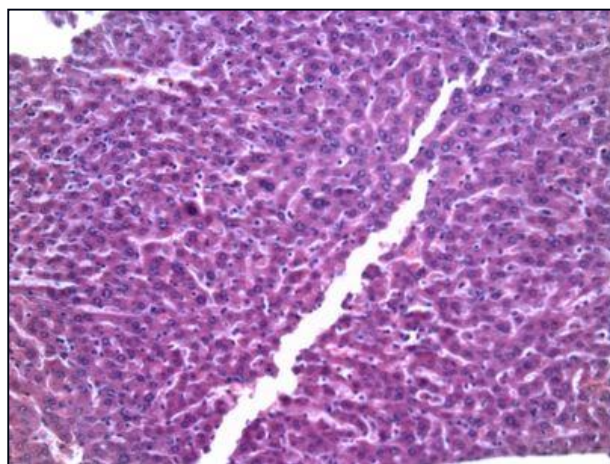
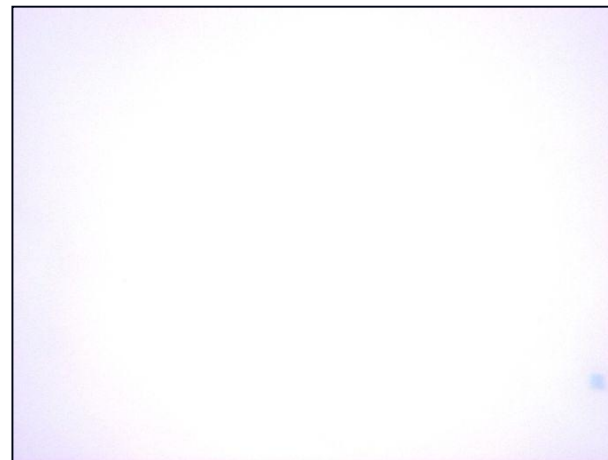
# 亮度校正实例

## ➤ 数字切片扫描系统



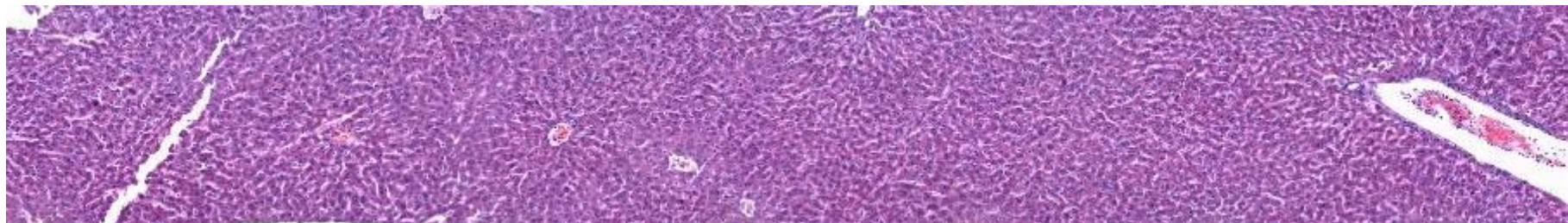
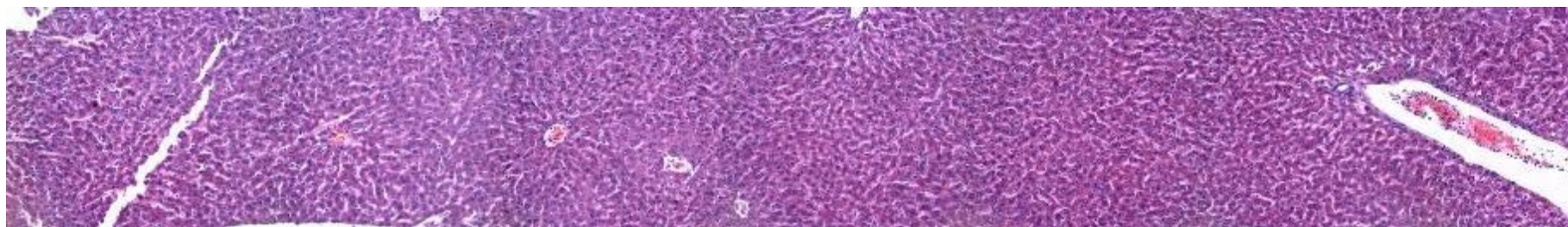
# 亮度校正实例

- 获取空白图像
- 按RGB三分量获取校正矩阵
- 图像亮度校正



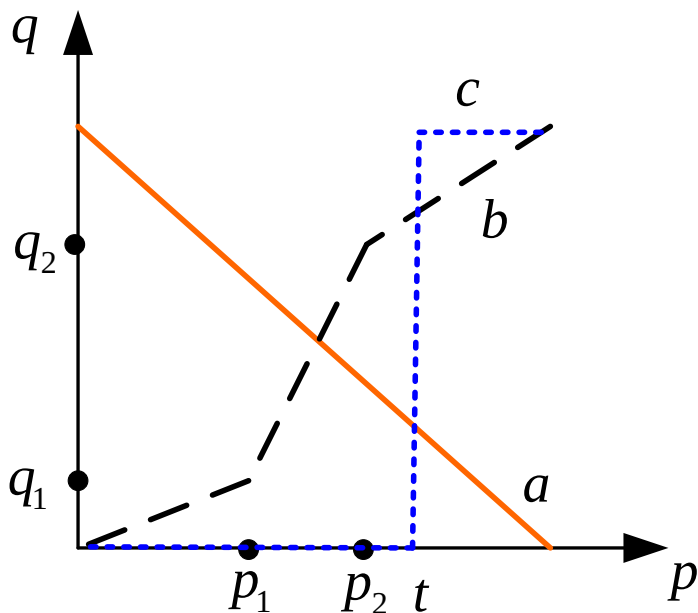
# 亮度校正实例

## ➤ 校正前后对比



## 3.1.2 灰度级变换

- 变换 $T$ ，将原来在范围 $[p_0, p_k]$ 内的亮度 $p$ 变换为一个新范围 $[q_0, q_k]$ 内的亮度  $q = T(p)$



- $a$ :底片变换
- $b$ :通过分段函数增强了图像在亮度 $p_1$ 到 $p_2$ 间的对比度
- $c$ :图像二值化

- 灰度级变换通常可以使用查表的方法实现

## 3.1.2 灰度级变换

- 分段增强
  - $[170, 200] \rightarrow [150, 225]$
- 二值化:  $t=100$



原始图像



底片变换



分段增强

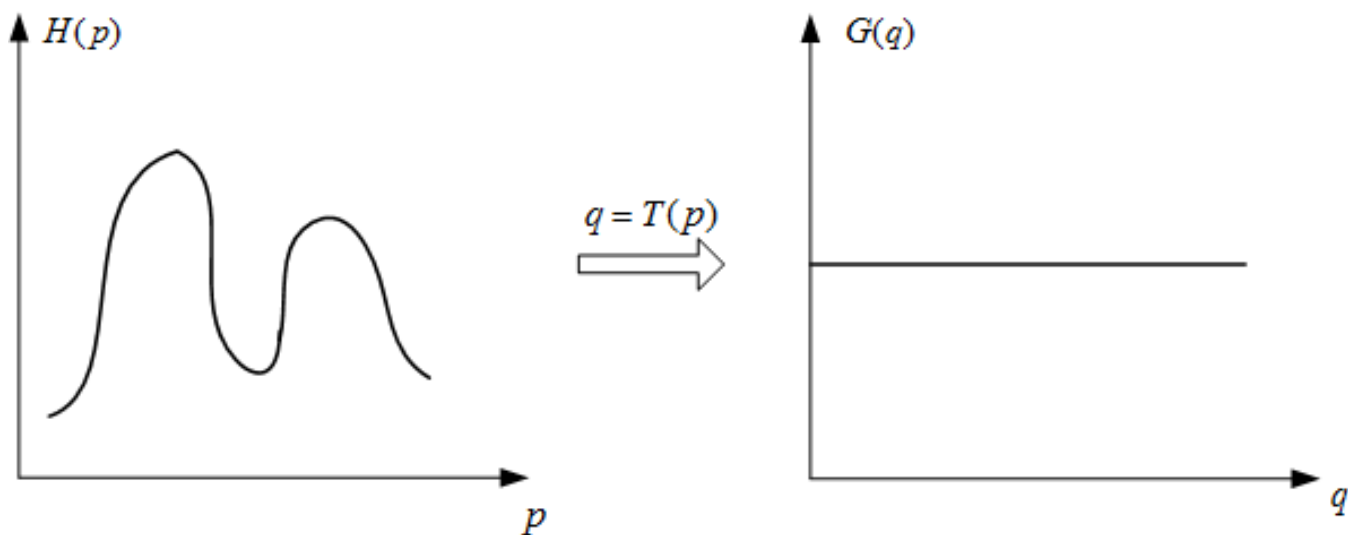


二值化

# 直方图均衡化(histogram equalization)



- 直方图均衡化的目标是创建一幅在整个亮度范围内具有相同亮度分布的图像
- 输入直方图 $H(p)$ ，输入亮度范围为 $[p_0, p_k]$ ，直方图均衡化的目标：找到一个单调的像素亮度变换 $q=T(p)$ ，使输出直方图 $G(q)$ 在整个输出亮度范围 $[q_0, q_k]$ 内是均匀的



# 直方图均衡化

- 直方图均衡化增强了靠近直方图极大值附近的亮度的对比度，减小了极小值附近的对比度
- 直方图可看作离散的概率密度函数，变换T是单调的

$$\sum_{i=0}^k H(p_i) = \sum_{i=0}^k G(q_i)$$

- 若图像有M行N列，均衡化后的直方图G(q)对应着均衡的概率密度函数，其函数值为一个常数

$$G(q_i) = \frac{MN}{q_k - q_0}$$

# 直方图均衡化

- 公式代入：当考虑“理想的”连续概率密度时，可以得到精确的均衡化直方图

$$\int_{p_0}^p H(s)ds = \int_{q_0}^q \frac{MN}{q_k - q_0} ds = \frac{MN(q - q_0)}{q_k - q_0}$$

- 得到像素亮度变换T

$$q = T(p) = \frac{(q_k - q_0)}{MN} \int_{p_0}^p H(s)ds + q_0$$

- 积分称为累计直方图，在数字图像中用求和近似，结果直方图并不是理想均衡的

$$q = T(p) = \frac{(q_k - q_0)}{MN} \sum_{s=p_0}^p H(s) + q_0$$

# 直方图均衡化

## 算法：直方图均衡化

1. 对于 $k(256)$ 个亮度级、大小为 $M \times N$ 的图像，创建长为 $k$ 的数组 $H$ ，初始化为0

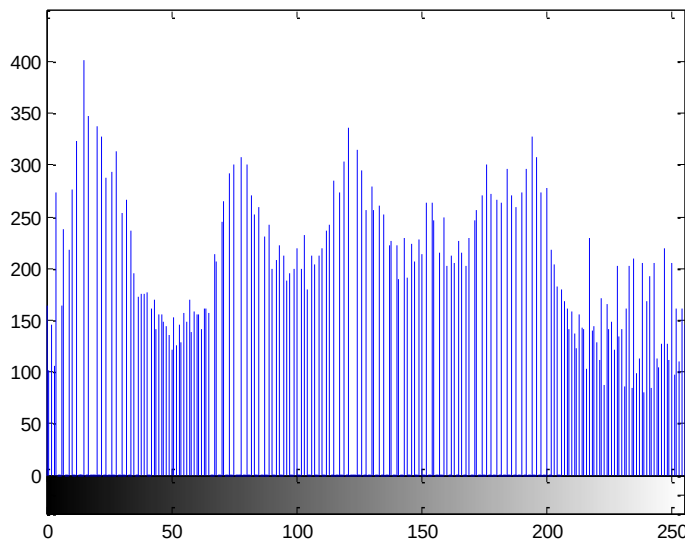
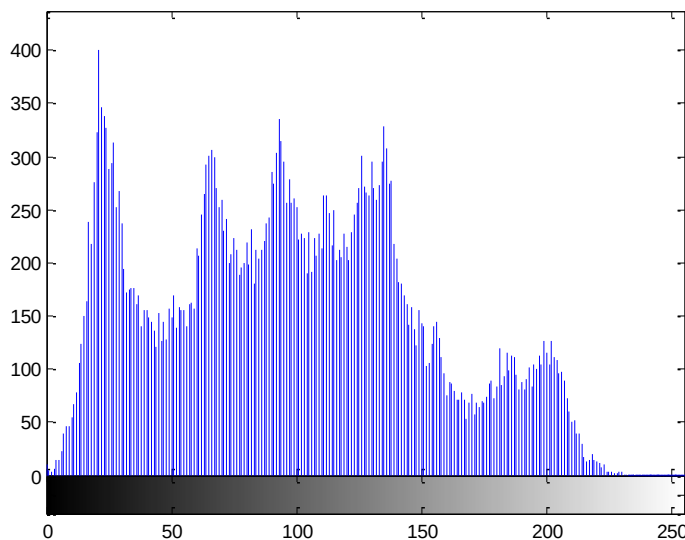
2. 形成图像直方图 $H$

3. 形成累计直方图 $H_c$   $H_c[0] = H[0]$

$$H_c[p] = H_c[p-1] + H[p] \quad p=1, 2, \dots, k-1$$

4. 设置  $T[p] = \text{round}(\frac{k-1}{MN} H_c[p])$ ,  $p = 0, 1, \dots, k-1$

5. 重新扫描图像，根据查找表获得变换结果



matlab: histeq

OpenCV:  
equalizeHist

## Matlab

imshow(i);

figure

imhist(i);

figure

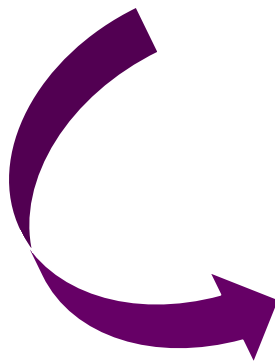
g = histeq(i,256);

imshow(g);

figure;

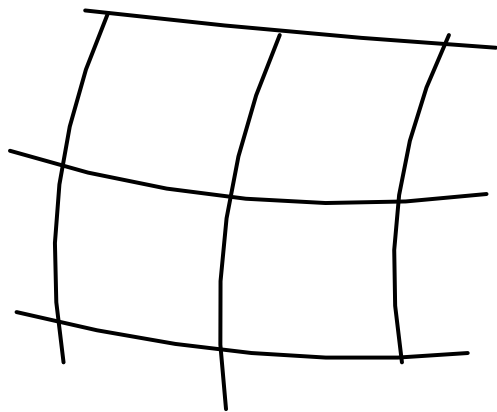
imhist(g);

# 思考：直方图均衡化



## 3.2 几何变换

- 几何变换用于消除图像采集时的几何畸变
- 几何变换的基本步骤
  - 1) 像素坐标变换：将输入图像的图像，输出点的坐标是连续数值
  - 2) 亮度插值：确定输出图像中整



### 七、科学施肥知识与技术

#### 1. 化肥与有机肥有什么主要的区别？

化肥与有机肥是当前最主要的两类肥料，它们都是庄稼的粮食。那么它们之间有什么主要区别呢？

首先，它们的来源不同。化肥又叫做化学肥料，是一类以矿物、空气、水为主要原料，通过物理、化学的方法制成的肥料。有机肥料通常又被叫做农家肥，泛指能用作肥料的各种有机物质，或称以有机物质为主要成分的肥料。有机肥料主要来自农村和城市的废弃物，包括种植业中的植物残体（秸秆、饼肥等）、养殖业中的畜禽粪尿以及人类粪和生活垃圾等。

其次，它们的肥性性能不同。化肥养分含量高，肥效快，便于储存和使用，但通常肥效时间短，肥力成分较单一，使用不当可能导致土壤结构变差以及环境污染。有机肥料肥效长，肥力成分丰富复杂，成本低，除了供给作物营养外，还可以改良培肥土壤，但养分含量相对较低，肥效缓慢。

#### 2. 化肥就是尿素吗？

当然不是。

由于尿素是生产实践当中应用最广泛、最常见的化肥，在提高产量方面曾起到巨大作用，以致很长一段时间以来，农村中很多朋友将尿素直接唤作化肥。这显然是不正确的。

化肥是所有化学肥料的总称，而尿素仅仅是化肥这个大家族中的一个成员，只是这个成员名气比较大而已。我们可以说“尿素是一种化肥”，但是不能说“化肥就是尿素”。这就好比我们可以说“成龙是一位演员”，但不能说“演员是成龙”一样。

#### 3. 什么是肥料三要素？

目前，国内外公认的农作物所必需的营养元素已经有 17 种，它们是碳、氢、氧、氮、磷、钾、钙、镁、硫、铁、硼、锰、铜、锌、钼、氯、镍。其中，镍是近年才发现的。

农作物生长需要的 17 种营养元素中，碳、氢、氧可以从空气和水中获得，一般不需要以肥料的形式提供。氮、磷、钾在作物体内含量较高，作物生长吸收得也较多，占干物重的百分之几到千分之几，称为“大量元素”。钙、镁、硫一般称为“中量元素”。铜、锌、铁、锰、硼、钼、氯、镍，作物需要量少，称为“微量元素”。氮、磷、钾三种元素通常都需要以肥料的形式补充，所以特别称它们为“肥料三要素”。

#### 4. 常用的化学氮肥、磷肥、钾肥各有哪些品种？

本地最常用的化学氮肥有尿素和碳酸氢铵。化学磷肥有过磷酸钙和钙镁磷肥。化学钾肥有氯化钾、硫酸钾。

## 3.2.1 像素坐标变换

- 像素坐标变换是一个矢量函数 $T$ ，将像素 $(x,y)$ 映射到新位置 $(x',y')$ :  $x'=Tx(x, y)$ ,  $y'=Ty(x, y)$
- 像素坐标变换通常用多项式公式近似:

$$x' = \sum_{r=0}^m \sum_{k=0}^{m-r} a_{rk} x^r y^k \quad y' = \sum_{r=0}^m \sum_{k=0}^{m-r} b_{rk} x^r y^k$$

- 若已知输入和输出图像中对应点对 $(x,y)$ 和 $(x',y')$ ，可通过求解线性方程组来确定 $a_{rk}$ 和 $b_{rk}$
- 通常需要比参数个数更多的点对以保证稳定性
- 对应点在图像中均匀分布，以更好地表达像素坐标变换
- 近似多项式的阶数越高，坐标变换对于对应点对的分布越敏感

# 举例



## ➤ SIFT特征检测



(a) Image 1



(b) Image 2



(c) SIFT matches 1



(d) SIFT matches 2

## 3.2.1 像素坐标变换

### ➤ 双线性变换(Bilinear Transform)

$$\begin{cases} x' = a_0 + a_1x + a_2y + a_3xy \\ y' = b_0 + b_1x + b_2y + b_3xy \end{cases}$$

- 确定变换系数需要四个对应点对

### ➤ 仿射变换(Affine Transform)

$$\begin{cases} x' = a_0 + a_1x + a_2y \\ y' = b_0 + b_1x + b_2y \end{cases}$$

- 确定变换系数需要三个对应点对
- 仿射变换包含了典型的几何变换，如平移、旋转、缩放

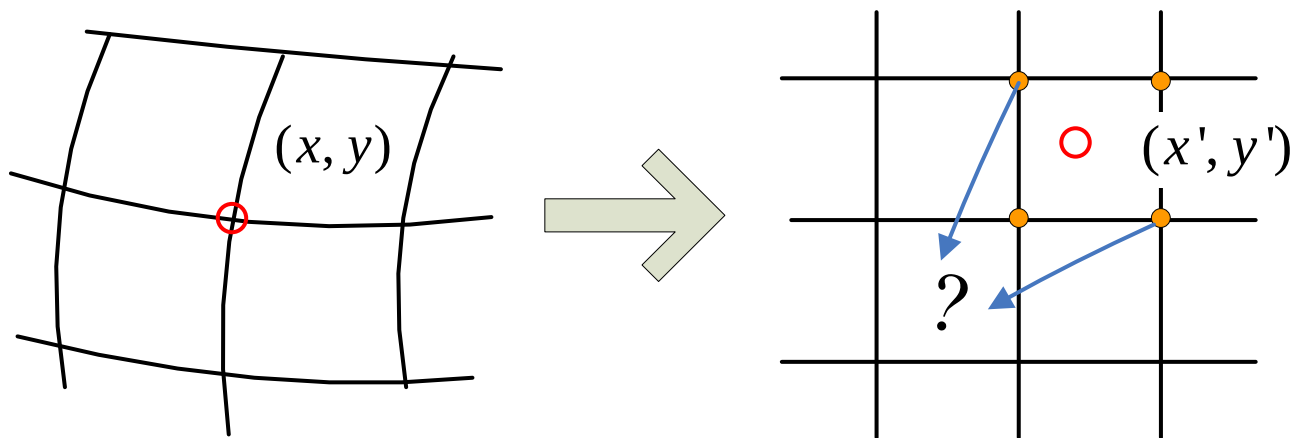
## 3.2.1 像素坐标变换

- 旋转变换：绕原点旋转角度 $\theta$ 
$$\begin{cases} x' = x \cos \theta + y \sin \theta \\ y' = -x \sin \theta + y \cos \theta \end{cases}$$
- 缩放变换：x轴为a，y轴为b
$$\begin{cases} x' = ax \\ y' = by \end{cases}$$
- 复杂的几何变换（失真、畸变）
  - 将图像分割成较小的矩形子图像
  - 对每个子图像，使用对应点对来估计一个简单的几何变换，如仿射变换
  - 在每个子图像中分别完成几何变换

## 3.2.2 亮度插值

### ➤ 进行亮度插值的原因

- 像素亮度变换将原始图像中的 $(x, y)$ 变换到了新位置 $(x', y')$ 一般说来，新坐标点的位置都是非整数
- 对于数字图像，需要得到离散栅格上的数值，输出图像栅格上每个像素的亮度值可通过邻域非整数采样点的亮度插值得到



## 3.2.2 亮度插值

### ➤ 进行亮度插值的原因

- 像素亮度变换将原始图像中的 $(x, y)$ 变换到了新位置 $(x', y')$ 一般说来, 新坐标点的位置都是非整数
- 对于数字图像, 需要得到离散栅格上的数值, 输出图像栅格上每个像素的亮度值可通过邻域非整数采样点的亮度插值得到

### ➤ 亮度插值

- 亮度插值影响图像品质, 插值越简单, 图像精度损失越大
- 常用插值方法: 最近邻插值、线性插值、双三次插值

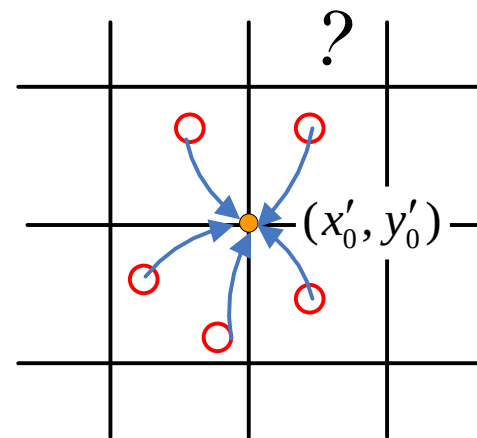
## 3.2.2 亮度插值

### ➤ 用对偶方法完成亮度插值

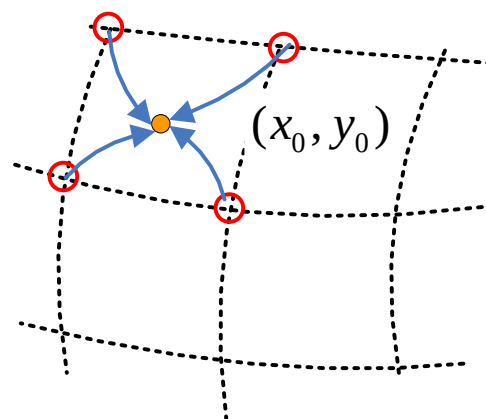
- 确定对应于输出图像离散栅格点在输入图像中原来点的亮度
- 假设计算在输出图像**离散栅格**上像素  $(x'_0, y'_0)$  的亮度值
- 输入图像中对应点  $(x_0, y_0)$  的坐标可通过像素坐标逆变换得到

$$(x_0, y_0) = \mathbf{T}^{-1}(x'_0, y'_0)$$

- 对输入图像进行亮度插值，获得  $(x_0, y_0) / (x'_0, y'_0)$  的亮度值



变换后结果



逆变换回原图

## 3.2.2 亮度插值

### ➤ 插值核

- 对于原图像 $f(l, k)$ ，其在 $(x, y)$ 点的亮度插值结果可以用卷积公式来表示：

$$f_n(x, y) = \sum_{l=-\infty}^{\infty} \sum_{k=-\infty}^{\infty} f(l, k) h_n(x - l, y - k)$$

$h_n$ 是插值核（interpolation kernel），一般只在很小的区域使用

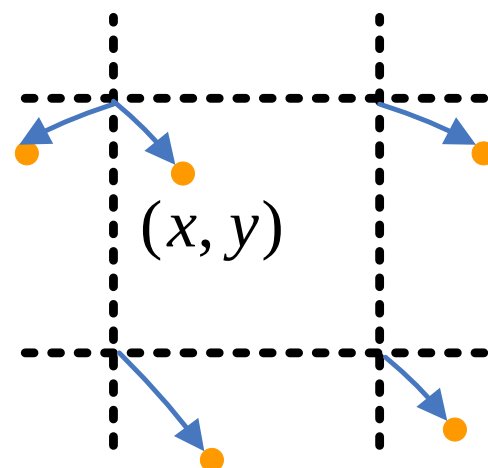
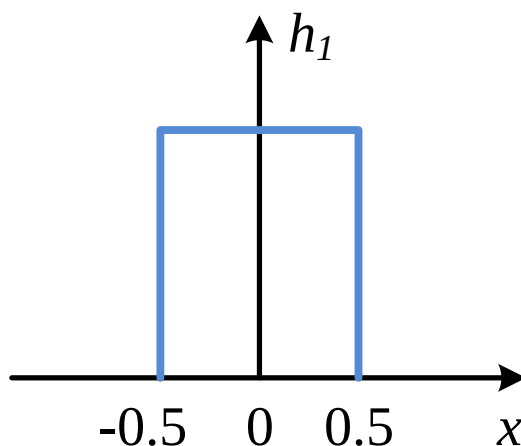
## 3.2.2 亮度插值

### ➤ 最近邻插值

- 赋值像素(x, y)的亮度值为栅格上最近像素点的亮度值

$$f_1(x, y) = f(\text{round}(x), \text{round}(y))$$

- 最近邻插值的位置误差最大是像素大小的一半, 变换后可能呈现阶梯状直边界



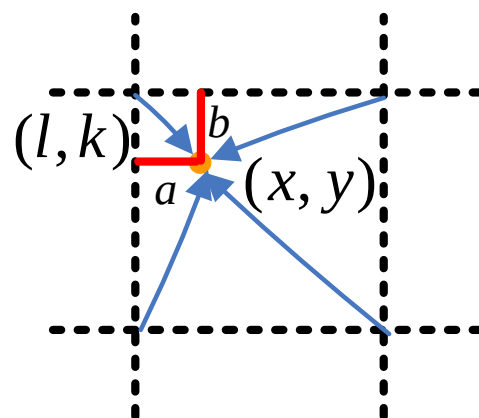
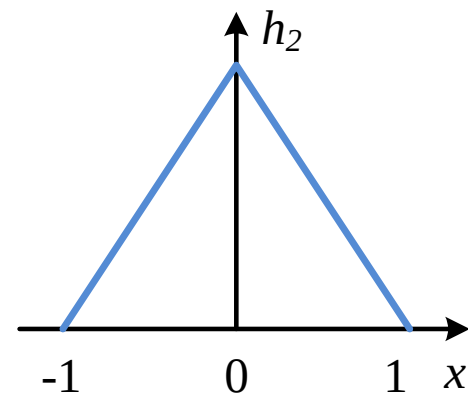
## 3.2.2 亮度插值

### ➤ 线性插值

- 考虑像素 $(x, y)$ 的四个相邻点，假设亮度在这个邻域内是线性的

$$\begin{aligned} f_2(x, y) &= (1-a)(1-b)f(l, k) & l &= \text{floor}(x) \\ &+ a(1-b)f(l+1, k) & k &= \text{floor}(y) \\ &+ b(1-a)f(l, k+1) & a &= x-l, b = y-l \\ &+ abf(l+1, k+1) \end{aligned}$$

- 线性插值可能引起较小的分辨率降低和模糊，但减轻了最近邻插值中出现的阶梯状直边界



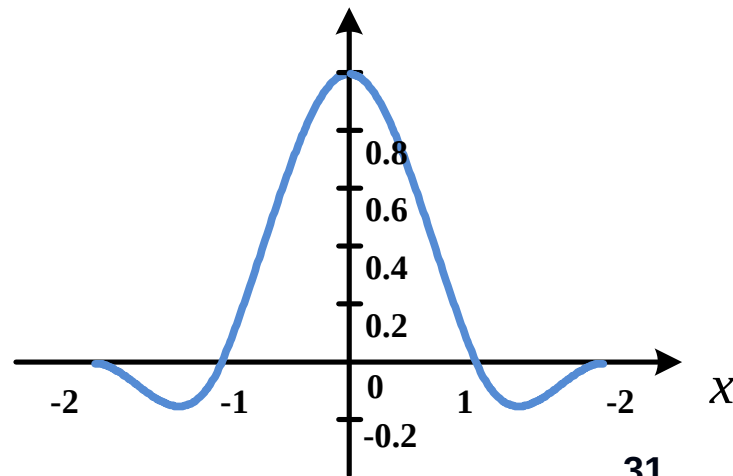
## 3.2.2 亮度插值

### ➤ 双三次插值

- 用双三次多项式局部近似亮度函数，考虑像素(x, y)的16个相邻点，一维插值核为：

$$h_3 = \begin{cases} 1 - 2|x|^2 + |x|^3 & 0 \leq |x| < 1 \\ 4 - 8|x| + 5|x|^2 - |x|^3 & 1 \leq |x| < 2 \\ 0 & \text{other} \end{cases}$$

- 双三次插值解决了阶梯状边缘问题和模糊问题，较好地保持了图像细节，常用于光栅显示



# 举例



(a) Image 1



(b) Image 2



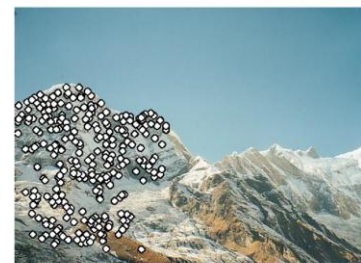
(c) SIFT matches 1



(d) SIFT matches 2



(e) RANSAC inliers 1



(f) RANSAC inliers 2

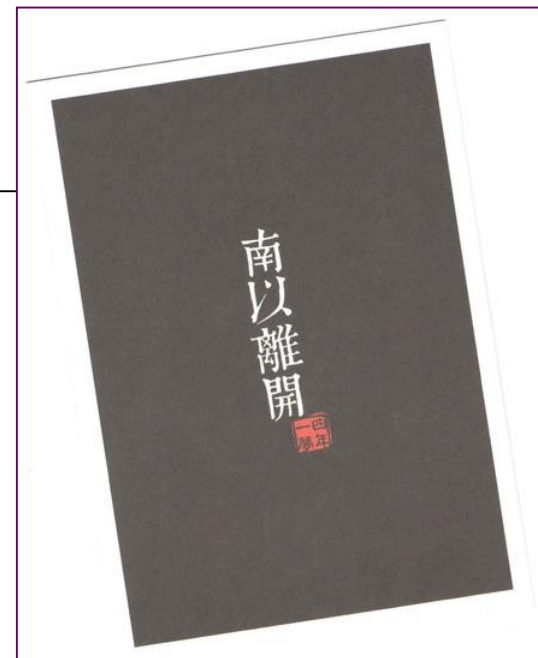


(g) Images aligned according to a homography

# 图像的旋转与缩放

## Matlab

```
rotateI = imrotate  
        (i,-10,'bilinear');  
resizeI = imresize  
        (rotateI,0.5);
```



## OpenCV

```
Point2f center = Point2f(img.cols / 2, img.rows / 2); // 旋转中心  
double angle = -10; // 旋转角度  
double scale = 0.5; // 缩放尺度
```

```
Mat rotateMat; // 构造仿射矩阵  
rotateMat = getRotationMatrix2D(center, angle, scale);
```

```
Mat rotateImg; // 实现图像的旋转和缩放  
warpAffine(img, rotateImg, rotateMat, img.size());
```

# 局部预处理

- **局部预处理**：使用输入图像中像素的一个**小邻域**内的像素信息，产生输出图像中新的对应像素的亮度值
- 这种预处理操作在信号处理领域又称为**滤波(filtering)**
- **依据处理目的分类**
  - 图像平滑(**Smoothing**)：目的是抑制噪声、提取感兴趣的特征，它利用了图像数据的冗余性
  - 梯度算子(**gradient operators**)：对图像函数计算局部导数
- **依据滤波变换的性质分类**
  - 线性滤波
  - 非线性滤波：基于邻域进行**非线性操作**，如中值滤波

# 线性滤波与非线性滤波

## ➤ 线性滤波

- 输出图像像素 $g(m,n)$ 的计算结果是输入图像像素 $f(m,n)$ 一个局部邻域 $\Omega$ 的亮度线性组合。邻域 $\Omega$ 中的像素贡献通过系数 $h$ 进行加权：

$$g(m,n) = \sum_{(k,l) \in \Omega} f(k,l)h(m-k,n-l) = f(m,n) * h(m,n)$$

$h$ ：滤波器、卷积掩模、核、窗口

- $\sum h = 1$ ，一般选择奇数行和奇数列的矩形邻域，容易确定邻域的中心。

## 3.3 图像平滑

### ➤ 3.3.1 均值滤波

- 假设每个像素上的噪声是一个均值为0，标准差为 $\sigma$ 的独立随机变量，则可通过多次采集相同的静态景物来获得一幅平均图像：

$$f(m, n) = \frac{1}{k} \sum_{i=1}^k f_i(m, n)$$

- 平均图像中的噪声仍是随机变量，均值为0，标准差为 $\frac{\sigma}{\sqrt{k}}$
- 若只能获得一幅带有噪声的图像，则通过图像的局部邻域平均完成滤波，这就是均值滤波
- 均值滤波属于线性滤波
- 如果噪声大小小于图像中感兴趣的最小尺寸，处理结果是可以接受的，但仍存在边缘模糊的问题

## 3.3.1 均值滤波

- 均值滤波是离散卷积的一个特例，对于 $3 \times 3$ 的邻域，卷积掩模 $h$ 为：

$$h = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

- 其他形式的卷积掩模：

$$h = \frac{1}{10} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$h = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

## 3.3.2 高斯滤波

- **高斯滤波**：基于高斯函数的形状形成卷积掩模，对于去除服从正态分布的噪声很有效

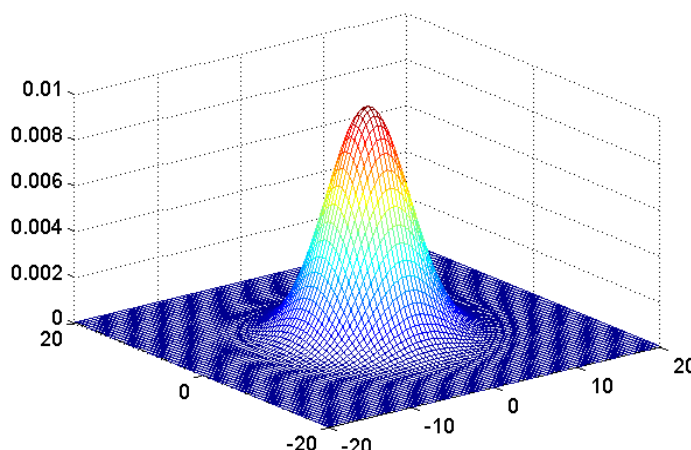
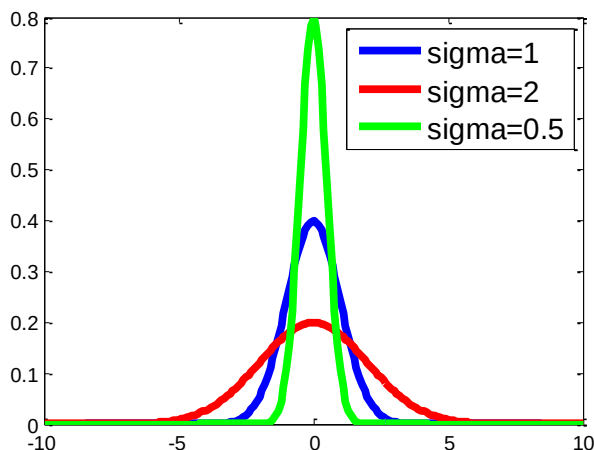
- **一维零均值高斯函数**

$$h(x, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2}{2\sigma^2}}$$

- **二维零均值高斯函数**

$$h(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

参数  $\sigma$  决定高斯函数的宽度，进而决定平滑程度



# 高斯函数的性质(1)

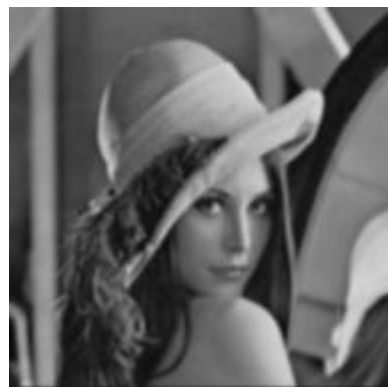
1. 高斯函数具有旋转对称性/各向同性，高斯滤波在各个方向上的平滑程度是相同的，从而使后续的边缘检测不会偏向某一方向
2. 高斯函数具有特殊的形状
3. 高斯函数的参数 $\sigma$  决定高斯函数的宽度，可通过调节 $\sigma$  在图像过于模糊（过平滑）与图像噪声/细纹理过多（欠平滑）之间进行折衷



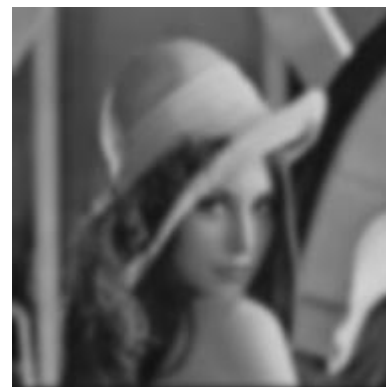
原始图像



$\sigma = 0.5$



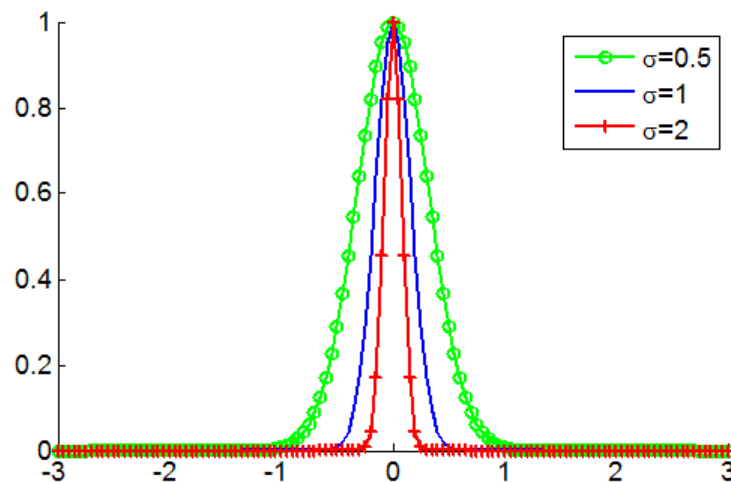
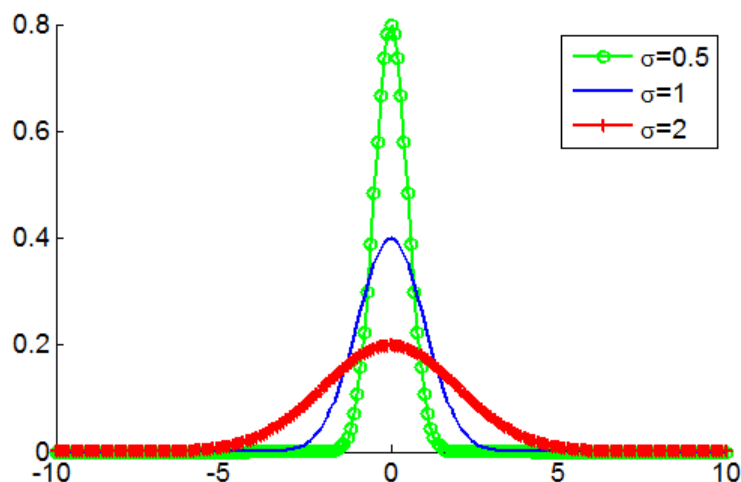
$\sigma = 1$



$\sigma = 2$

# 高斯函数的性质 (2)

## 4. 高斯函数的傅立叶变换结果仍是高斯函数



- 高斯滤波器是一种低通滤波器，图像中的噪声一般是高频信号，高斯滤波可以在一定程度上抑制较高频率的噪声，同时保留大部分所需的低频信号

# 高斯函数的性质 (3)

## 5. 高斯函数的可分离性

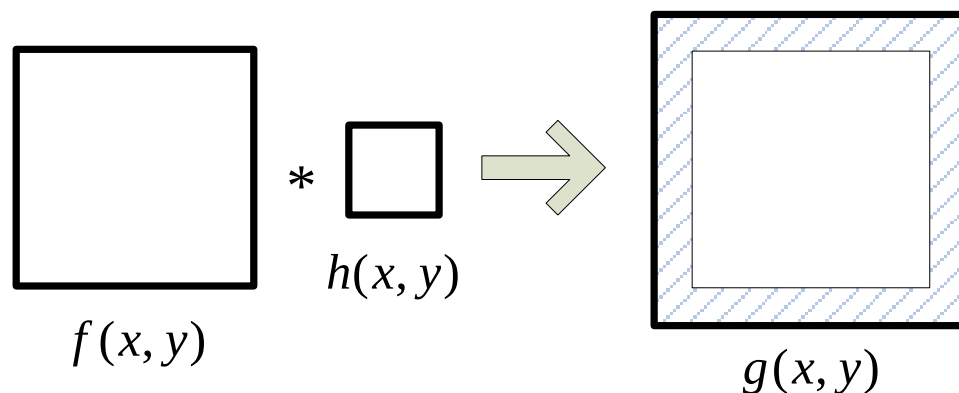
$$\begin{aligned} f(m, n) * h(m, n) &= \sum_{k=0}^{M-1} \sum_{l=0}^{N-1} h(k, l) f(m-k, n-l) \\ &= \sum_{k=0}^{M-1} \sum_{l=0}^{N-1} \frac{1}{2\pi\sigma^2} e^{-\frac{k^2+l^2}{2\sigma^2}} f(m-k, n-l) \\ &= \frac{1}{2\pi\sigma^2} \sum_{k=0}^{M-1} e^{-\frac{k^2}{2\sigma^2}} \left[ \sum_{l=0}^{N-1} e^{-\frac{l^2}{2\sigma^2}} f(m-k, n-l) \right] \end{aligned}$$

- 二维高斯函数的卷积可以分两步进行：首先图像与水平方向一维高斯函数进行卷积，然后将卷积结果与垂直方向相同的一维高斯函数卷积
- 二维高斯滤波的计算量随模板宽度成线性增长

# 线性滤波函数

## ➤ Matlab函数

- `conv2`: 计算二维卷积



- `imfilter`: 多尺度图像滤波，通过边界选项和输出尺寸选项得到不同的输出效果
  - 边界选项: X, 'symmetric', 'replicate', 'circular'
  - 输出尺寸选项: 'same', 'full'
- `filter2`: 二维数字滤波

# 线性滤波函数

## ➤ Matlab函数

- `conv2`: 计算二维卷积
- `imfilter`: 多尺度图像滤波，通过边界选项和输出尺寸选项得到不同的输出效果
  - 边界选项: X, 'symmetric', 'replicate', 'circular '
  - 输出尺寸选项: 'same', 'full '
- `filter2`: 二维数字滤波

## ➤ OpenCV函数

- `blur`: 均值滤波
- `GaussianBlur`: 高斯滤波
- `filter2D`: 线性滤波，用'replicate'填充边界

# Matlab

```
i = imread('miss.bmp');
i = i(:,:,1);
imshow(i);
figure;
in = imnoise(i, 'gaussian', 0,
0.001);
imshow(in);
figure;
if1 = imfilter(in, ones(3,3)/9,
'conv','replicate');
imshow(if1);
figure;
if2 = imfilter(in, ones(7,7)/49,
'conv','replicate');
imshow(if2);
```



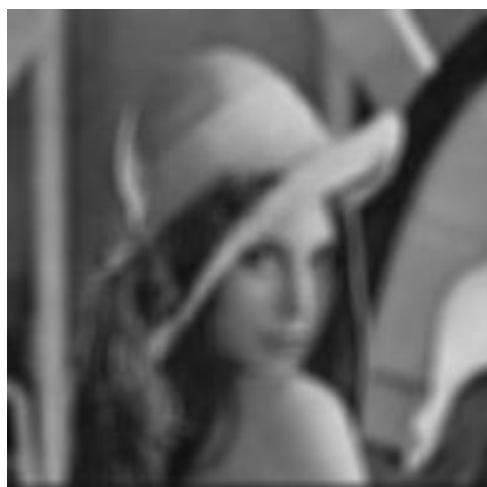
原始图像



加噪声结果



**3 × 3**均值掩模



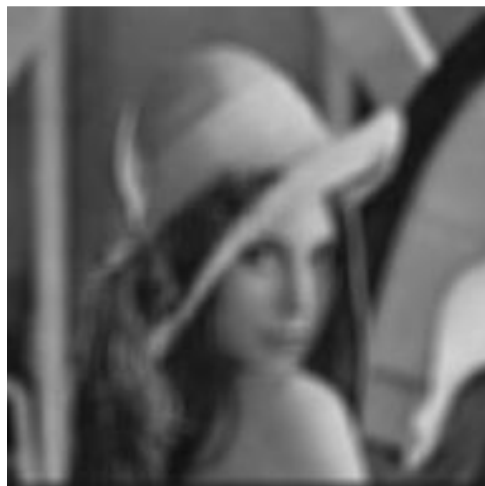
**7 × 7**均值掩模

# Matlab

```
h1 = fspecial
    ('gaussian',3,0.5);
gf1 = imfilter(in,h1, 'conv',
    'replicate');
imshow(gf1);
figure;
h2 = fspecial('gaussian',7,1);
gf2 = imfilter(in,h2, 'conv',
    'replicate');
imshow(gf2);
```



**3×3均值掩模**



**7×7均值掩模**



**3×3高斯掩模**



**7×7高斯掩模**

# OpenCV(C++)

```
// 使用filter2D进行均值滤波 (3*3掩膜)
Mat f1(img.size(), img.type());
Mat k1 = Mat::ones(Size(3, 3), CV_32F);
filter2D(noise, f1, -1, k1/9);

// 使用blur函数进行平均 (7*7掩膜)
Mat f2(img.size(), img.type());
blur(noise, f2, Size(7, 7));

// 使用GaussianBlur函数进行高斯滤波
Mat gf1(img.size(), img.type());
GaussianBlur(noise, gf1, Size(3, 3), 0.5);
Mat gf2(img.size(), img.type());
GaussianBlur(noise, gf2, Size(7, 7), 1);
```

# OpenCV(Python)

```
# 创建高斯噪声图像
noise = np.random.normal(0, 10, img.shape).astype(np.uint8)
noise = img + noise # 将噪声添加到原图像

# 使用filter2D进行均值滤波 (3x3掩膜)
k1 = np.ones((3, 3), dtype=np.float32) / 9 # 创建3x3均值滤波核
f1 = cv2.filter2D(noise, -1, k1) # 应用滤波

# 使用blur函数进行平均滤波 (7x7掩膜)
f2 = cv2.blur(noise, (7, 7)) # 7x7均值滤波

# 使用GaussianBlur函数进行高斯滤波
gf1 = cv2.GaussianBlur(noise, (3, 3), 0.5) # 3x3高斯滤波, 标准差为0.5
gf2 = cv2.GaussianBlur(noise, (7, 7), 1) # 7x7高斯滤波, 标准差为1
```

## 3.3.3 非线性滤波

- 线性平滑的问题：图像的边缘被模糊了
- 非线性平滑方法：根据指定标准，将当前像素邻域分成两个子区域，仅使用邻域中与被处理像素有类似性质的点完成平滑
- 非线性滤波
  - 限制数据有效性下的平均
  - 中值滤波

# 1) 限制数据有效性下的平均

- 仅使用满足某种标准的像素做平均，从而避免模糊
  - 设定非法数据范围[min,max]
  - 只有在[min,max]内的像素值才被其邻域的平均所取代
  - 只有有效的数据才对邻域的平均有贡献
  - 卷积掩模：

$$h(i, j) = \begin{cases} 1 & f(m+i, n+j) \notin [\min, \max] \\ 0 & \text{other} \end{cases}$$



## 2) 中值滤波

- **基本思想：**用像素点邻域灰度值的中值来代替该像素点的灰度值
- **中值滤波的步骤：**
  - 1) 确定掩模大小和掩模中心
  - 2) 在掩模内将像素点按亮度值大小排序
  - 3) 选择序列的中间值作为掩模中心的新像素值
  - 如果像素点数为偶数，中值取排序像素中间两点的平均值
- 中值滤波不依赖于邻域内与典型值差别很大的灰度值，可以减少边缘的模糊，对去除**脉冲噪声**、**椒盐噪声**非常有效

# Matlab

```
i = imread('miss.bmp');
i = i(:,:,1);
imshow(i);
figure
in = imnoise(i, 'salt &
pepper', 0.1);
imshow(in);
figure;
if1 = medfilt2(in,[2 2],
'symmetric');
imshow(if1);
figure;
if2 = medfilt2(in,[3 3],
'symmetric');
imshow(if2);
```



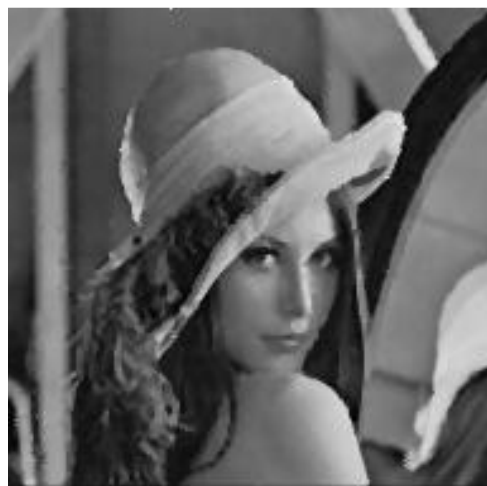
原始图像



加椒盐噪声



$2 \times 2$  掩模



$3 \times 3$  掩模

# OpenCV(C++)

```
double salt_pepper = 0.1;    // 修改像素所占比例

// 产生椒盐噪声
srand(time(NULL)); // 获取种子
double randNo;

Mat noise = img.clone();
for(int i=0; i<noise.rows; i++)    // 行
    for(int j=0; j<noise.cols; j++) // 列
    {
        randNo = double(rand())/RAND_MAX;
        if(randNo>1-salt_pepper/2) // 变为白色像素
            noise.at<uchar>(i, j) = 255;
        else if(randNo<salt_pepper/2) // 变为黑色像素
            noise.at<uchar>(i, j) = 0;
    }

// 使用medianBlur函数进行中值滤波
Mat mf(noise.size(), noise.type());
medianBlur(noise, mf, 3);
```

# OpenCV(Python)

# 产生椒盐噪声

```
noise = np.copy(img) # 复制原始图像
```

```
salt_count = int(salt_pepper * noise.size / 2) # 盐噪声像素数量
```

```
pepper_count = salt_count # 椒噪声像素数量
```

# 随机选择盐噪声和椒噪声的像素位置

```
salt_coords = [np.random.randint(0, i - 1, salt_count) for i in noise.shape]
```

```
pepper_coords = [np.random.randint(0, i - 1, pepper_count) for i in noise.shape]
```

```
noise[salt_coords[0], salt_coords[1]] = 255 # 添加盐噪声
```

```
noise[pepper_coords[0], pepper_coords[1]] = 0 # 添加椒噪声
```

# 使用中值滤波

```
mf = cv2.medianBlur(noise, 3)
```

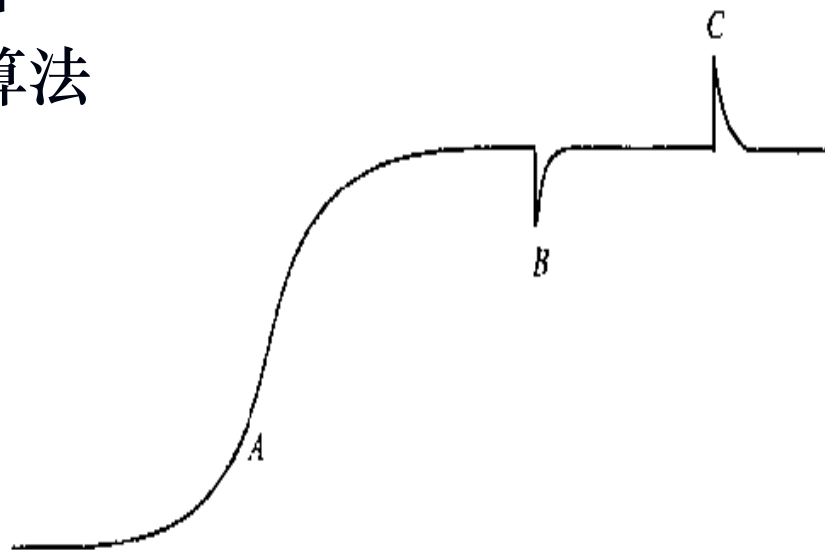
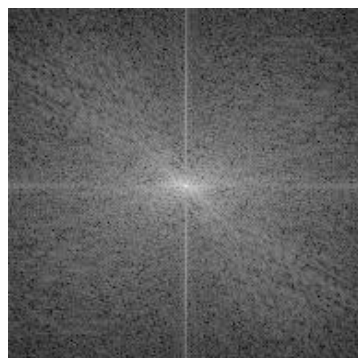
## 3.4 边缘检测

- 边缘的重要性：边缘点包含了图像中的大多数信息
- 边缘点的物理意义
  - 空间曲面的不连续点
  - 不同材料或不同颜色产生的边缘点
  - 物体与背景的分界线——外轮廓线
  - 阴影引起的边缘
- 边缘是图像上灰度的不连续点，或者是灰度变化剧烈的地方



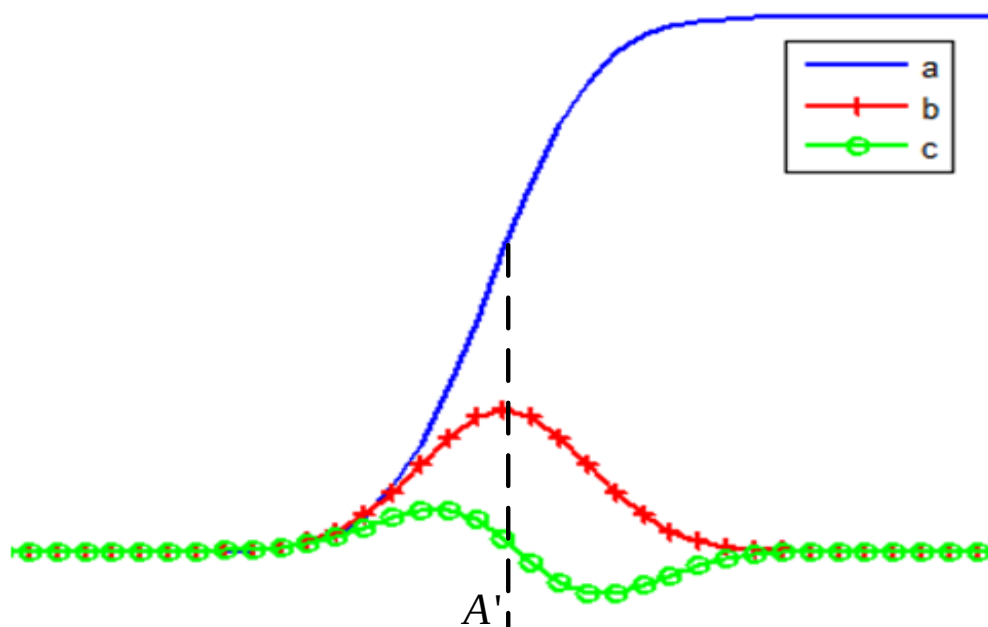
## 3.4 边缘检测

- 在边缘点附近图像具有频域的高频分量，所有边缘检测方法在本质上都是检测信号的高频
- 边缘检测难点：区分图像自身的高频分量和噪声
- 边缘检测
  - 3.4.1 边缘检测与滤波器
  - 3.4.2 Canny边缘检测算法



## 3.4.1 边缘检测与滤波器

- 边缘是图像上灰度的不连续点，或者是灰度变化剧烈处
- 边缘的数学表达：信号一阶微分最大值/两阶微分过零点



- a: 原始信号
- b: 一阶微分
- c: 二阶微分

## 3.4.1 边缘检测与滤波器

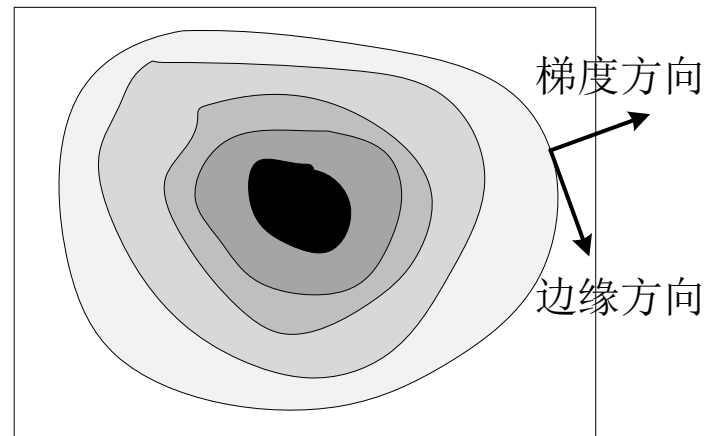
- 使用梯度(gradient)描述图像函数的变化，梯度方向是图像函数增长最大的方向

- 对于二维图像 $f(x, y)$ ，梯度定义为：

$$\nabla f = \left[ \frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]^T$$

- 梯度的幅值： $|\nabla f| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$

- 梯度的方向： $\psi = \arg\left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right)$



## 3.4.1 边缘检测与滤波器

- 图像一阶微分是梯度的模：

$$|\nabla f| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

- 图像二阶微分应理解为沿梯度方向的二阶方向导数，计算比较复杂，一般采用两阶微分算子（拉普拉斯算子）表示，拉普拉斯算子具有各向同性

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

- 各向同性：对图像各个方向的作用是相同的，具有旋转不变性

# 连续情况：微分滤波器

## ➤ 噪声对边缘检测的影响

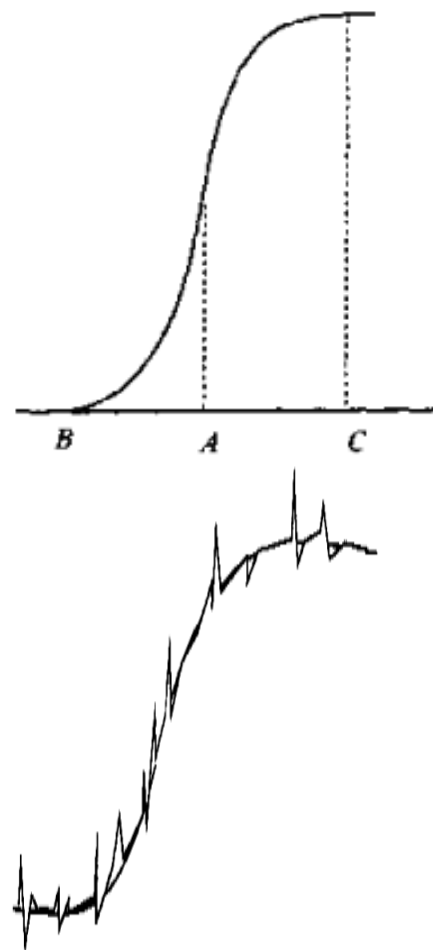
- 实际信号都是有噪声的，按照一阶微分最大值或二阶微分过零点的方法检测边缘，可能检测出**噪声引起的假边缘**

## ➤ 解决方法

- 先对原始图像 $f(x,y)$  进行**平滑滤波**：

$$g(x, y) = f(x, y) * h(x, y)$$

- 再对滤波后的信号 $g(x,y)$ 求一阶或两阶微分以检测边缘



# 连续情况：微分滤波器

## ➤ 微分滤波器

- 微分运算与卷积运算次序可以互换

$$\begin{aligned} g'(x) &= \frac{d(f(x) * h(x))}{dx} = \frac{d}{dx} \int_{-\infty}^{\infty} f(s)h(x-s)ds \\ &= \int_{-\infty}^{\infty} f(s)h'(x-s)ds = f(x) * h'(x) \end{aligned}$$

- 可将先平滑、后微分的两步运算合并，直接求  $h'$  或  $h''$
- $h'$  称为一阶微分滤波器， $h''$  称为两阶微分滤波器

## ➤ 边缘检测的基本方法

- 设计平滑滤波器  $h$ ，计算  $h'$  和  $h''$
- 检测  $f * h'$  的局部最大值或的  $f * h''$  的过零点

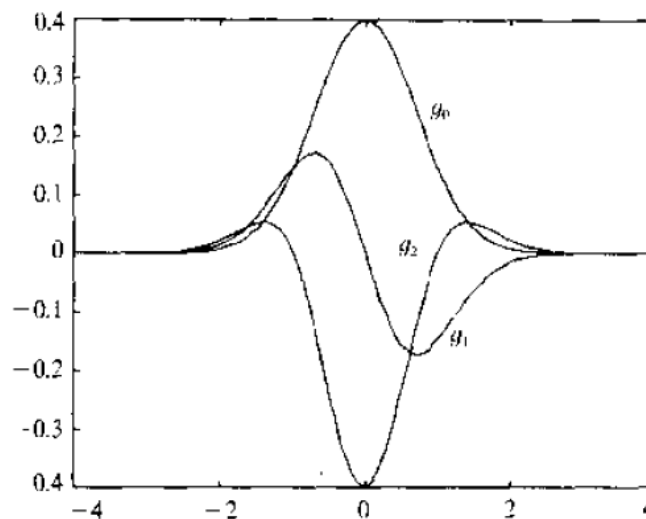
# 连续情况：微分滤波器

- 平滑滤波器 $h(x)$ 应满足以下条件：
  - 当 $|x| \rightarrow \infty$ 时， $h(x) \rightarrow 0$ ，且 $h(x)$ 为偶函数
  - $\int_{-\infty}^{+\infty} h(x) dx = 1$
  - $h(x)$ 一阶及二阶可微
- 常用的平滑滤波器：高斯函数（一维）

$$h(x, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2}{2\sigma^2}}$$

$$h'(x, \sigma) = \frac{-x}{\sqrt{2\pi}\sigma^3} e^{-\frac{x^2}{2\sigma^2}}$$

$$h''(x, \sigma) = \frac{1}{\sqrt{2\pi}\sigma^3} e^{-\frac{x^2}{2\sigma^2}} \left( \frac{x^2}{\sigma^2} - 1 \right)$$



# 连续情况：微分滤波器

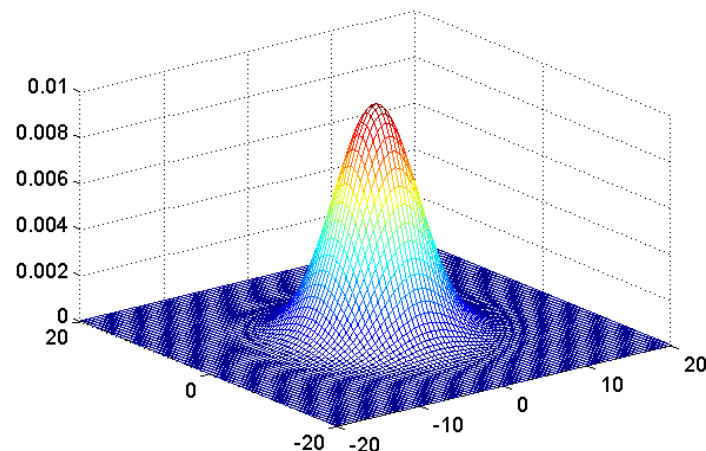
## ➤ 二维高斯平滑滤波器

$$h(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

## ➤ 一阶微分滤波器

$$\frac{\partial h(x, y, \sigma)}{\partial x} = \frac{-1}{\sqrt{2\pi}\sigma} e^{-\frac{y^2}{2\sigma^2}} \frac{x}{\sqrt{2\pi}\sigma^3} e^{-\frac{x^2}{2\sigma^2}}$$

$$\frac{\partial h(x, y, \sigma)}{\partial y} = \frac{-1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2}{2\sigma^2}} \frac{y}{\sqrt{2\pi}\sigma^3} e^{-\frac{y^2}{2\sigma^2}}$$



## ➤ 二阶微分滤波器（高斯-拉普拉斯算子）

$$\nabla^2 h(x, y, \sigma) = \frac{1}{2\pi\sigma^4} \left( \frac{x^2 + y^2}{\sigma^2} - 2 \right) e^{-\frac{x^2+y^2}{2\sigma^2}}$$

# 离散情况：差分滤波器

## ➤ 连续到离散

| 连续   | 离散   |
|------|------|
| 积分运算 | 求和运算 |
| 微分运算 | 差分运算 |

## ➤ 离散信号的平滑滤波用离散卷积表示

$$\begin{aligned} g(m, n) &= f(m, n) * h(m, n) \\ &= \sum_{(k,l)} f(k, l) h(m - k, n - l) \end{aligned}$$

# 离散情况：差分滤波器

## ➤ 离散信号的一阶差分运算

➤ 前向差分:  $f^{(1)}(n) = f(n+1) - f(n)$

➤ 后向差分:  $f^{(1)}(n) = f(n) - f(n-1)$

➤ 前后向平均差分:  $f^{(1)}(n) = (f(n+1) - f(n-1))/2$

## ➤ 离散信号的二阶差分运算

➤ 二阶差分是一阶差分的差分

$$f^{(2)}(n) = f(n+1) + f(n-1) - 2f(n)$$

## ➤ 图像的一阶差分运算

➤ 水平方向:  $f_x(i, j) = f(i, j) - f(i, j-1)$

➤ 垂直方向:  $f_y(i, j) = f(i, j) - f(i-1, j)$

# 高斯差分滤波器

## ➤ 常用的平滑滤波器：高斯函数（一维）

$$h(n) = ce^{-\frac{n^2}{2\sigma^2}}$$

- c称为归一化常数
- 与连续平滑滤波器一样， $\sum_{-\infty}^{+\infty} h(n) = 1$  以保证滤波后不改变原信号的均值
- 连续的高斯函数 $h(x)$ ，当 $|x| > 3$ 时， $h(x) \approx 0$ ；离散情况下， $h(n)$ 在 $-3\sigma \leq n \leq 3\sigma$ 内取样
- 若整数 $n$ 的取值范围为 $-N \leq n \leq N$ ，则 $h(n)$ 可用一个长度为 $2N+1$ 的窗口表示
- 一阶/二阶差分滤波器：对高斯平滑滤波器求差分

# 一阶差分滤波器

## ➤ Robert算子

$$h_1 = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}, h_2 = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

## ➤ Prewitt算子

$$h_1 = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}, h_2 = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

## ➤ Sobel算子

$$h_1 = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}, h_2 = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

# Sobel算子

$$h_1 = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}, h_2 = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

OpenCV: Sobel



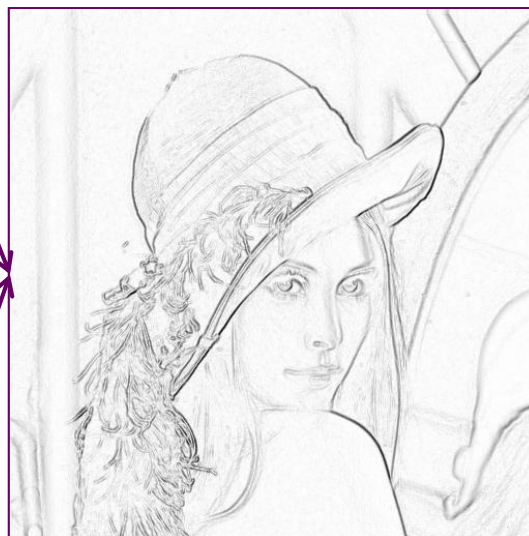
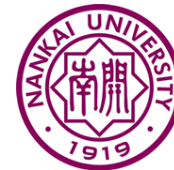
水平(X)方向平滑  
+ 垂直方向差分



垂直(Y)方向平滑  
+ 水平方向差分

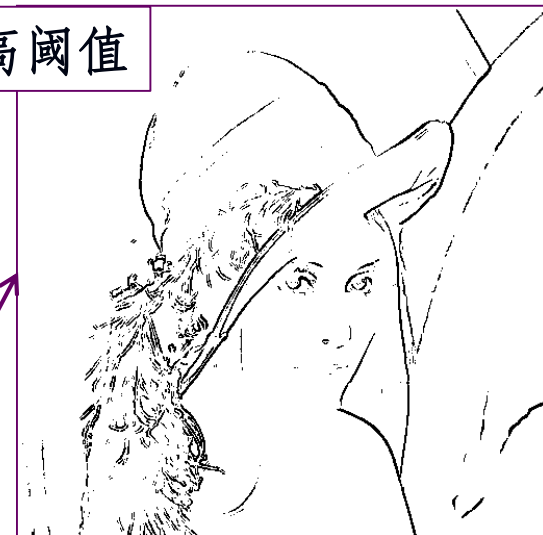
# 使用Sobel算子检测边缘

边缘图像  
國值化方法



$\text{abs}(\text{sobelX})$   
 $+\text{abs}(\text{sobelY})$

高國值



低國值





原始图像



Roberts算子



Prewitt算子



Sobel算子

## Matlab

```
i = imread('miss.bmp');  
i = i(:,:,1);  
figure;  
ro = edge(i,'roberts');  
imshow(1-ro);  
figure;  
pre = edge(i,'prewitt');  
imshow(1-pre);  
figure;  
so = edge(i,'sobel');  
imshow(1-so);
```

# 二阶差分滤波器

## ➤ 拉普拉斯算子

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

$$\frac{\partial^2 f}{\partial y^2} \sim \Delta_y^2 f(m, n)$$

$$= \Delta_x [f(m+1, n) - f(m, n)]$$

$$= f(m+2, n) - 2f(m+1, n) + f(m, n)$$

$$\sim f(m+1, n) - 2f(m, n) + f(m-1, n)$$

$$\frac{\partial^2 f}{\partial x^2} \sim \Delta_x^2 f(m, n)$$

$$\sim f(m, n+1) - 2f(m, n) + f(m, n-1)$$

$$h = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

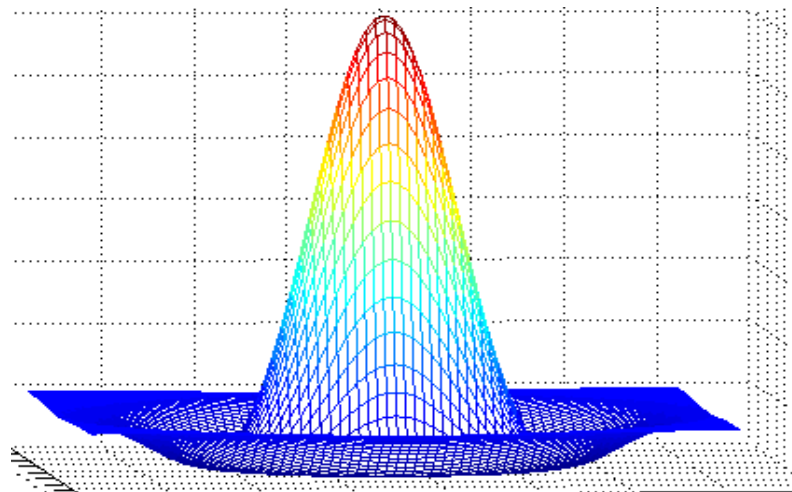
# 二阶差分滤波器

$$h = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

## ➤ 高斯-拉普拉斯算子(LoG)

$$\nabla^2 (f * h) = f * \nabla^2 h \quad \nabla^2 h(x, y) = \frac{1}{2\pi\sigma^4} \left( \frac{x^2 + y^2}{\sigma^2} - 2 \right) e^{-\frac{x^2 + y^2}{2\sigma^2}}$$
$$\sim c \left( \frac{x^2 + y^2 - 2\sigma^2}{\sigma^4} \right) e^{-\frac{x^2 + y^2}{2\sigma^2}}$$

|   | 1      | 2       | 3       | 4       | 5      |
|---|--------|---------|---------|---------|--------|
| 1 | 0.0239 | 0.0460  | 0.0499  | 0.0460  | 0.0239 |
| 2 | 0.0460 | 0.0061  | -0.0923 | 0.0061  | 0.0460 |
| 3 | 0.0499 | -0.0923 | -0.3182 | -0.0923 | 0.0499 |
| 4 | 0.0460 | 0.0061  | -0.0923 | 0.0061  | 0.0460 |
| 5 | 0.0239 | 0.0460  | 0.0499  | 0.0460  | 0.0239 |



# 使用拉普拉斯算子检测边缘

OpenCV:Laplacian

$$h = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$



**3\*3核**



**7\*7核**



检测过零点作为边缘

# 使用LoG算子检测边缘



## Matlab

```
i = imread('miss.bmp');  
i = i(:,:,1);  
figure;  
log = edge(i,'log',[],0.5);  
imshow(1-log);  
figure;  
log = edge(i,'log',[],1);  
imshow(1-log);  
figure;  
log = edge(i,'log',[],2);  
imshow(1-log);  
figure;  
log = edge(i,'log',[],3);  
imshow(1-log);
```



$\sigma = 0.5$



$\sigma = 1$



$\sigma = 2$



$\sigma = 3$

# 滤波器的尺度



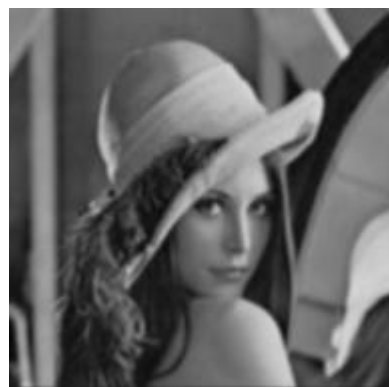
- 用不同参数的高斯滤波器检测边缘， $\sigma$ 越大，图像分辨率越低，检测到的边缘点越少



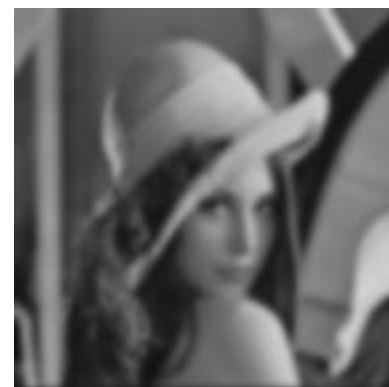
原始图像



$\sigma = 0.5$



$\sigma = 1$



$\sigma = 2$



# 滤波器的尺度

- 用不同参数的高斯滤波器检测边缘， $\sigma$ 越大，图像分辨率越低，检测到的边缘点越少
- 滤波器的尺度影响边缘检测结果
- 边缘检测的困难：**边缘的定义不明确**



原始图像



$\sigma = 0.5$

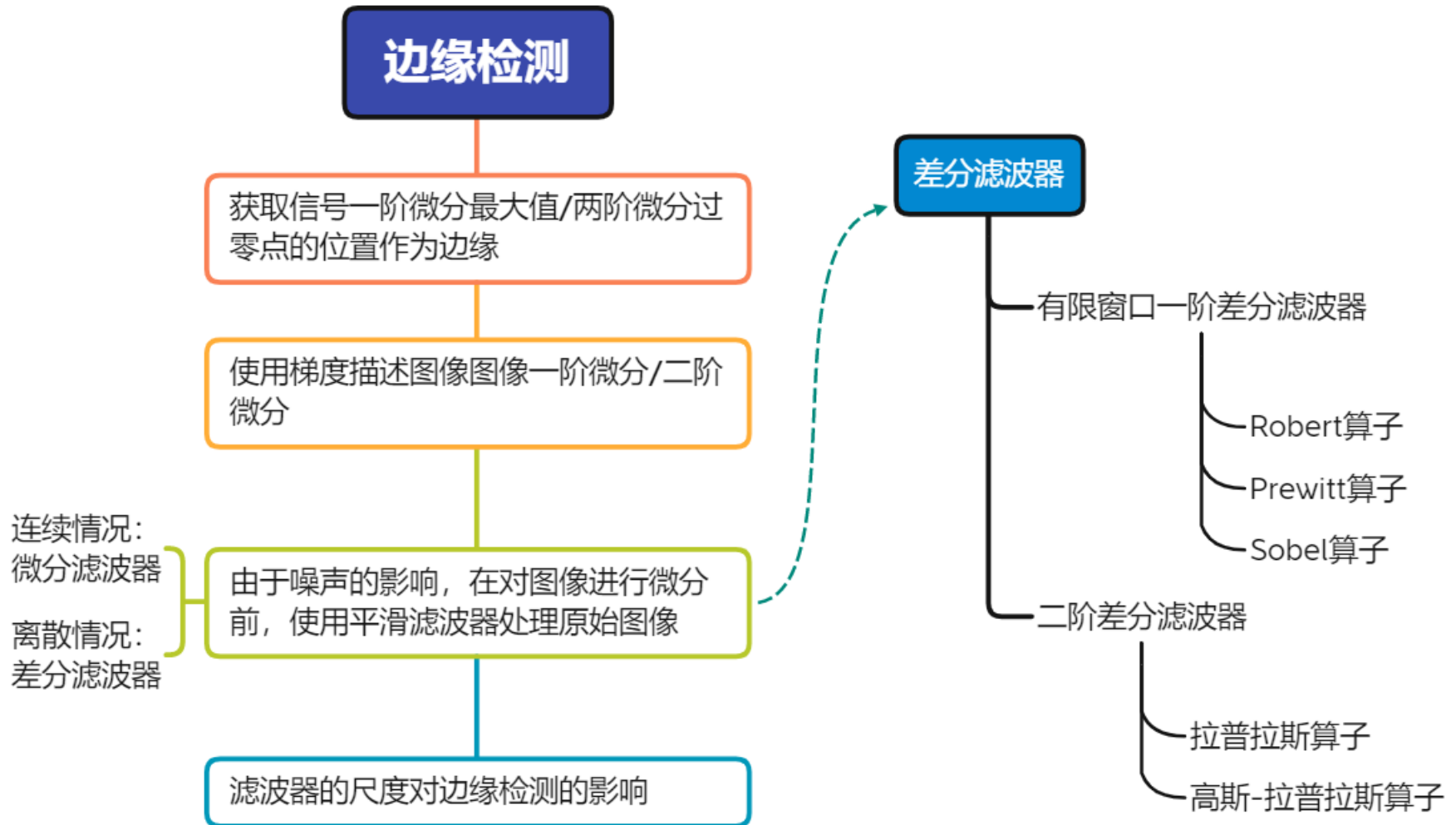


$\sigma = 1$



$\sigma = 2$

# 总结: 3.4.1 边缘检测与滤波器



## 3.4.2 Canny边缘检测

### ➤ 边缘模型：阶跃边缘

$$f(x) = \begin{cases} A, & x > x_0 \\ 0, & x \leq x_0 \end{cases}$$



### ➤ 噪声模型：可加性的白噪声

$$\bar{f}(x) = f(x) + \eta(x)$$

$\eta(x)$  均值为零、方差为  $\varepsilon^2$  的白噪声

<https://www2.eecs.berkeley.edu/Faculty/Homemepages/canny.html>

### ➤ 滤波器：有限脉冲响应滤波器

Canny J. A Computational Approach to Edge Detection[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1986, PAMI-8(6):679-698.

# 边缘检测的Canny准则

## ➤ Canny最优化准则

- 检测标准（最大信噪比准则，Good detection）：要有好的检测结果，不丢失重要的边缘，不应有虚假的边缘
- 定位标准（最优过零点准则，Good localization）：实际边缘位置与检测到的边缘位置间的偏差最小
- 单响应标准（多峰值响应准则，Only one response to a single edge）：对实际上的同一边缘要有低的响应次数

# 边缘检测的Canny准则

- Canny以一维形式为例，给出了三条准则的数学表达式

$$\text{SNR} = \frac{\left| \int_{-W}^{+W} G(-x) f(x) dx \right|}{n_0 \sqrt{\int_{-W}^{+W} f^2(x) dx}} \quad \text{Localization} = \frac{\left| \int_{-W}^{+W} G'(-x) f'(x) dx \right|}{n_0 \sqrt{\int_{-W}^{+W} f'^2(x) dx}}.$$

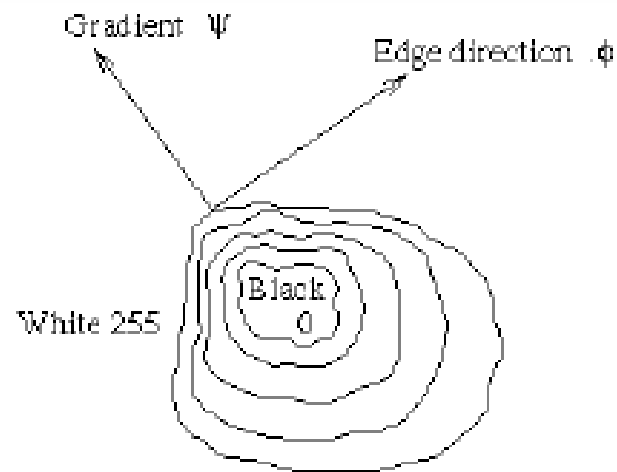
检测标准

定位标准

- 单响应标准—— Eliminating Multiple Responses
- 寻找最优滤波器的问题转换为泛函的约束优化问题

# Canny准则下的最优边缘检测滤波器

- 针对一维信号和前两个最优化准则，可微积分方法可以得到滤波器的完整解，加入第三个最优化准则，则需通过数值优化方法求解
- 最优滤波器可近似为标准差为 $\sigma$ 的高斯一阶微分滤波器，误差小于20%
- 对于二维情况，阶跃边缘由位置、方向、幅度确定
- 将图像与二维高斯函数做卷积再沿梯度方向微分，就构成了一个简单而有效的滤波器



# Canny边缘检测算法(1)

## ➤ 边缘检测滤波器 将图像与二维高斯函数做卷积再沿梯度方向微分

- 设G是尺度为 $\sigma$ 的高斯函数， $G_n$ 是G沿方向 $\bar{n}$ 的一阶方向导数：

$$G_n = \frac{\partial G}{\partial \bar{n}} = \bar{n} \nabla G$$

- 对图像f中的每个像素，局部边缘的法向 $\bar{n}$ 可估计为：

$$\bar{n} = \frac{\nabla(G * f)}{|\nabla(G * f)|}$$

- 找到边缘位置，边缘位于 $G_n$ 与图像f的卷积在方向 $\bar{n}$ 的局部最大值处：

$$\frac{\partial}{\partial \bar{n}} G_n * f = \frac{\partial^2}{\partial \bar{n}^2} G * f = 0$$

- 计算边缘强度：  $|G_n * f| = |\nabla(G * f)|$

# Canny边缘检测算法 (2)

## ➤ 双阈值技术

边缘检测通过阈值化确定突出的边缘，若滤波结果超出或低于阈值，则噪声引起的单边缘虚假响应会使检测出的边缘不连续，Canny通过滞后阈值化处理解决该问题：

- 图像的响应大于高阈值，它一定是边缘
- 图像的响应小于低阈值，它一定不是边缘
- 图像的响应在高低阈值之间，如果它与大于高阈值的像素相连，它也可能是边缘
- 高、低阈值可根据对信噪比的估计确定

# Canny边缘检测算法(2)

## ➤ 双阈值技术



**Sobel**低阈值  
边缘检测结果



**Sobel**高阈值  
边缘检测结果



**Canny**边缘检测  
(双阈值)

# Canny边缘检测算法 (3)

## ➤ 多尺度技术

- 滤波器的尺度选择一直是边缘检测的一大难题
- 对于Canny边缘检测，不同的尺度对应着不同方差的高斯操作

## 特征综合方法

- 标记出所有由最小尺度算子得到的突出边缘
- 把小尺度算子的边缘检测结果与高斯函数作用，估计较大尺度滤波器检测该边缘的位置（大尺度合成边缘响应）
- 将相同的较大尺度滤波器直接作用于原图像，得到边缘检测结果（大尺度实际边缘响应）
- 如果实际边缘响应比合成预测的响应显著强时，才接受这个边缘点

# Canny边缘检测算法

算法：Canny边缘检测

1. 将图像 $f$ 与尺度为 $\sigma$ 的二维高斯函数 $h$ 做卷积以消除噪声

$$g = f * h(\sigma)$$

2. 对 $g$ 中的每个像素计算梯度的大小和方向

$$|\nabla g| = \sqrt{g_x^2 + g_y^2} = \sqrt{\left(\frac{\partial g}{\partial x}\right)^2 + \left(\frac{\partial g}{\partial y}\right)^2}$$

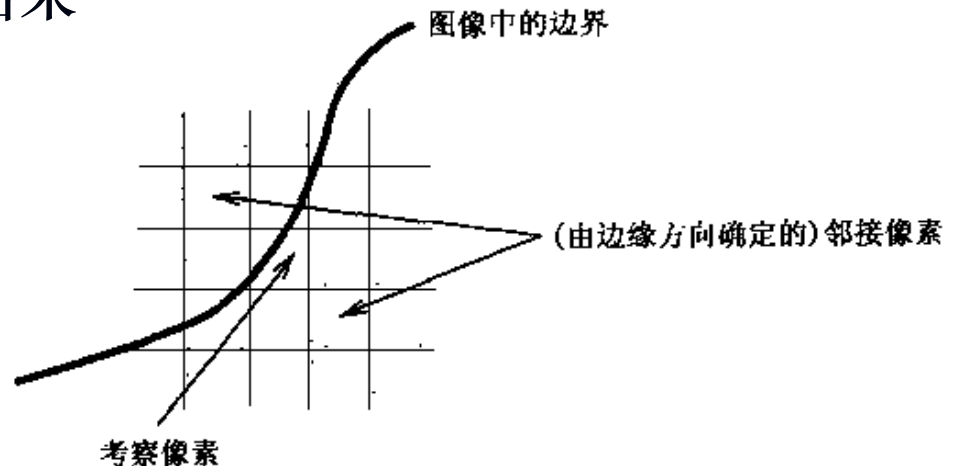
$$\theta = \arctan\left(\frac{g_y}{g_x}\right)$$

# Canny边缘检测算法

## 算法：Canny边缘检测

### 3. 根据像素**梯度方向**，通过非极大值抑制获取边缘位置

- 遍历 $|\nabla g|$ ，对于每个非0幅值的像素，考察由梯度方向指出的两个邻接像素
- 如果两个邻接像素的幅值有一个超过当前考察像素的幅值，则将当前考察像素标记出来
- 处理边缘图像上的所有像素，重新扫描图像，将标记像素置为0



# Canny边缘检测算法

算法：Canny边缘检测（续）

4. 计算得到的 $|\nabla g|$ 是边缘强度
5. 滞后阈值化处理
  - 设置高阈值 $t_1$ 和低阈值 $t_0$
  - 将幅值超过 $t_1$ 的所有边缘标注为正确边缘（强边缘）
  - 扫描幅值在区间 $[t_0, t_1]$ 内的所有像素
  - 如果该像素与已标注为边缘的另一像素接壤，将它也标记出来
  - 从第3步重复直到稳定
6. 对于递增的标准差 $\sigma'$ ，重复上述步骤1—5
7. 用特征综合方法，收集来自多尺度的最终边缘信息

## Matlab: edge



**edge(i,'canny')**

$$\sigma = \sqrt{2}$$

低閾值: 18

高閾值: 44

**edge(i,'canny',[],2.5)**

$$\sigma = 2.5$$

低閾值: 26

高閾值: 64

## Matlab: edge



## OpenCV: Canny



$$\sigma = \sqrt{2}$$



$$\sigma = 2.5$$

# 总结



# The End



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南開大學

Nankai University